SignBridge

Industrial Inclusion Through Computer Vision

Real-time sign-language recognition for human–robot and human–human communication on the factory floor

Final Presentation — Industrial Applications of Computer Vision — A.A. 2025–26
University of Padua
Stanis Brian · Faakhir · Giovanni Camerotto

Agenda

What we'll cover

01 Problem & Vision

02 System Pipeline & Sensors

03 System in Action

04 Dataset, Ground Truth & Training

05 Model Architectures

06 Results & Confusion Matrix

07 Embedded Deployment (Raspberry Pi)

08 Repository, References & Next Steps

The Human-Centric Problem

01 · Vision & Scope

The Challenge

Industrial environments rely heavily on verbal commands or complex touch interfaces. Deaf and speech-impaired workers face a “second barrier” when interacting with autonomous cobots or hearing teammates.

430M+

people worldwide live with disabling hearing loss

The SignBridge Vision

A computer-vision system that closes this dual gap in Industry 5.0: it translates sign language in real time to command robots, and it converts signs into text / speech for hearing colleagues.

By 2030, robots will be everywhere on the factory floor — they must be intelligent enough for everyone.

Three Objectives, One System

01 · Vision & Scope

Human–Robot Bridge

Translates hand gestures directly into robot commands for real-time cobot control and safety stops.

Human–Human Bridge

Converts sign language into text-to-speech output for hearing colleagues — instant collaborative feedback.

Industrial Safety

Provides visual and audio feedback to the worker, ensuring safety parity across the whole team.

System Pipeline

02 · From camera frame to action

1	**Acquisition**	Single monocular RGB webcam (laptop) captures live video frames.
2	**Hand Detection + Landmark Regression**	MediaPipe Hands — a pre-trained deep CNN — detects the hand region and regresses 21 keypoints per frame, end-to-end. Stacked across a short time window, this becomes a 2D-position-plus-time signal. This replaces classical segmentation and hand-crafted feature extraction (no CLAHE / manual filtering / thresholding needed).
3	**Feature Normalization**	Keypoints are converted to coordinates relative to the wrist — reduces sensitivity to where the hand appears in the frame.
4	**Classification**	Static-sign classifier (hand shape) or point-history classifier (FC / CNN / Transformer) assigns a gesture class from the short landmark-trajectory window.
5	**Decision & Output**	Recognized label drives on-screen feedback, a robot command (ROS), and/or text-to-speech for hearing colleagues.

Sensors & Runtime Setup

02 · What hardware is actually behind this

Camera

Single laptop-integrated RGB webcam — no depth sensor, no IR, no external camera rig.

Compute (prototype)

Laptop CPU only — no dedicated GPU / AI accelerator used for the results in this deck.

Measured throughput

~9 FPS end-to-end on this laptop (see live screenshot, FPS:9.07) — the working prototype's real measured number, not a target.

Why this matters for the Raspberry Pi section: this 9 FPS laptop baseline is what we compare embedded-hardware throughput against later in the presentation.

Problem Definition & Metrics

02 · Being explicit about scope, as required

Input / Output / Out of Scope

Input: a short window of hand-landmark coordinates from a single RGB camera.

Output: one of 5 dynamic gesture classes (plus 4 additional static hand-shape classes), mapped to a robot command or a spoken phrase.

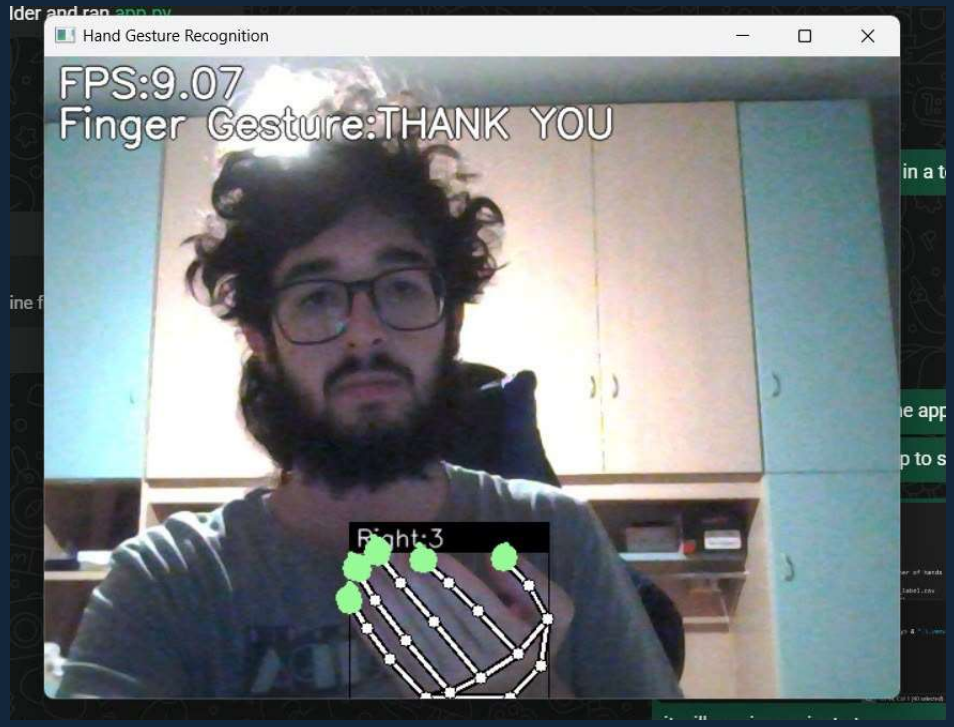
Out of scope: full sentence-level sign language translation, multi-hand simultaneous signing beyond our tracked hands, and non-manual markers (facial expression).

Performance Metrics — and Why

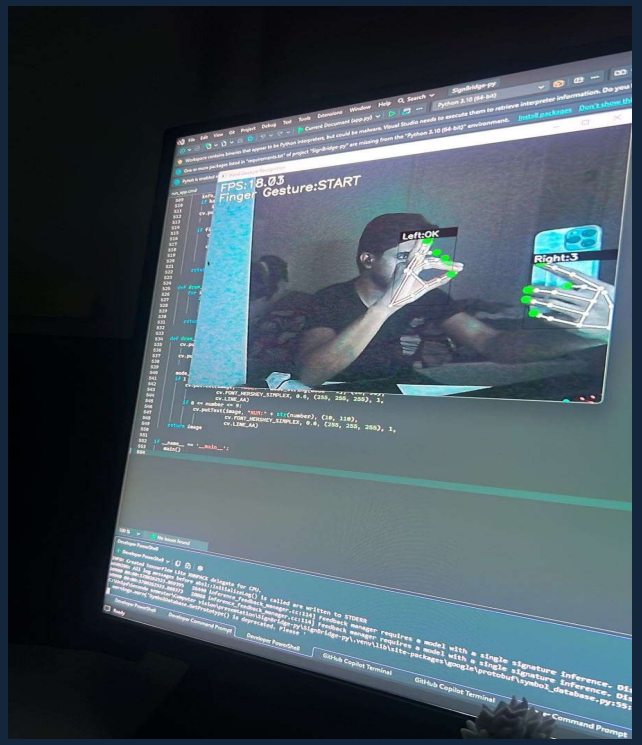
- Accuracy — overall share of correctly classified gestures.
- Precision / Recall / F1 per class — to catch confusions between visually similar signs.
- Macro & weighted averages — checks performance isn't hiding behind a dominant class.
- These are standard, widely-used classification metrics — chosen so our results are directly comparable to other work, not a custom scoring scheme.

System in Action

03 · Live captures from two team members' setups



Static sign — "THANK YOU", single hand



Dynamic gesture — "START", dual-hand tracking (Left: OK · Right: 3)

Captured on two different laptops / team members — keeps the working demo honest about who and what hardware it was tested on, not just one person's setup.

Dataset & Ground Truth

04 · Built from scratch, as required

How data was captured

- "Logging" mode in our app: pressing a number key (0–9) tags the current frame's landmark vector with that class ID and appends it as a row to a CSV file.
- Static hand-shape samples → keypoint.csv (21 landmarks × x,y).
- Dynamic gesture samples → point_history.csv (fingertip trajectory over a short frame window).
- This keypress-tagged label is the ground truth — no external annotation tool needed, but every sample is a real, verified capture of the intended sign.

The Sign Lexicon

Category	Gestures	Type	Action / Feedback
Safety	EMERGENCY, STOP	Dynamic	Immediate E-Stop / lock joints
Control	START, PLEASE	Dynamic	Initialize routine / call supervisor
Feedback	OK, 1, 2, 3	Static	Task confirmation / multi-step select
Polite	THANK YOU	Dynamic	Text-to-speech for colleagues

9 signs total: 5 dynamic gestures (evaluated with FC/CNN/Transformer) + 4 static hand shapes. 75% / 25% train–validation split, fixed random seed · 63 held-out samples used to evaluate the dynamic-gesture models · baseline = simplest architecture (Fully-Connected).

Gesture Difficulty: Easy / Medium / Hard

04b · Answering the professor's scope question with real evidence

Our categorization

EASY

Static hand shapes: OK, 1, 2, 3

Clearly distinct handshapes with no motion component — the professor's own definition of 'easy.'

MEDIUM

Dynamic, distinct-motion gestures: START, PLEASE, EMERGENCY

Involve motion, but each has a visually distinct trajectory from the others.

HARD

THANK YOU and STOP

Empirically confusable — confirmed below, not assumed.

The evidence, not just intuition

During early testing, the system did not reliably distinguish THANK YOU from STOP — confirmed by cross-class errors appearing off the diagonal in our confusion matrices. This is empirical evidence of confusability, not a guess based on how the signs look.

Sign-language phonology literature analyzes every sign along a small set of parameters: handshape, location, movement, and palm orientation (Stokoe, 1960; extended by Battison, 1978; Liddell & Johnson, 1989). Signs that share most of these parameters — called minimal pairs — are known to be harder to visually discriminate. Our THANK YOU / STOP confusion is consistent with this: they likely share more of these parameters with each other than with the rest of our 5-gesture set.

How the Models Were Trained

04 · Same recipe, three architectures

Optimizer

Adam

Adaptive learning rate; used for all three models, unchanged.

Loss function

Sparse categorical cross-entropy

Standard choice for single-label multi-class classification with integer class IDs.

Split

75% train / 25% validation

Fixed random seed — identical split reused for every architecture, for a fair comparison.

Regularization

Dropout (0.2 – 0.5)

Applied after the input and before the final dense layers to reduce overfitting on a small dataset.

Stopping rule

Early stopping on val_loss

Training halts once validation loss stops improving — checkpoint of the best epoch is kept.

Epoch budget

Up to 1000 epochs

In practice all three models stopped early well before that cap.

Grounding Our Technical Choices in the Literature

05b · Why Adam, cross-entropy, and attention aren't arbitrary picks

Optimizer — Adam

Adaptive Moment Estimation combines per-parameter adaptive learning rates with momentum, needing little manual tuning.

Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980.

Loss function — categorical cross-entropy

The standard loss for multi-class classification with a softmax output layer; minimizing it is equivalent to maximizing the likelihood of the correct class.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

Our overall design pattern, in the literature

Our pipeline — MediaPipe landmarks feeding separate static- and dynamic-gesture classifiers, benchmarked across CNN and Transformer architectures — matches the approach taken in recent published work on assistive hand-sign recognition. This supports our architectural strategy as a legitimate, actively-studied approach. We have not verified that this source's own CNN-vs-Transformer numbers match our specific accuracy ordering, so we are not claiming external validation of that result — only of the design pattern itself.

"A hand sign recognition based signal system for mute people using machine learning" — MediaPipe-based hand tracking with twofold static/dynamic classifiers, benchmarked against CNN, Transformer, and TinyML baselines, for the same assistive-communication purpose.

Model Comparison — Overview

05 · Three architectures, same data, same split

Fully-Connected

Baseline

Treats the whole time window as one flat vector — no notion of time order.

CNN (1D)

Spatial-temporal

Learns local motion patterns across nearby frames via 1D convolutions.

Transformer

Attention-based

Learns which frames matter most via self-attention, regardless of distance.

Full layer-by-layer diagrams follow on the next three slides.

Same input window, same 75/25 split, same optimizer (Adam) and loss (sparse categorical cross-entropy), same 5-class label set, early stopping on validation loss — the only variable is the architecture.

Architecture — Fully-Connected (Baseline)

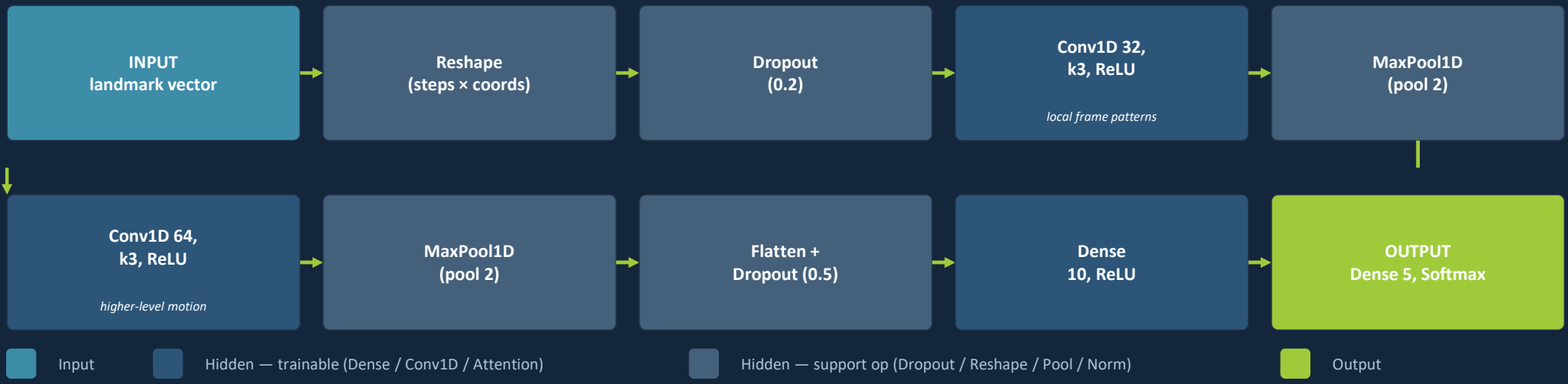
05 · Layer-by-layer: 1 input · 4 hidden · 1 output

Why it's the baseline

Simplest possible model: it has no way to represent “which frame happened first” — every timestep and coordinate is just another number in one long input vector, so any temporal structure must be learned implicitly, if at all, by the two hidden Dense layers. Result: 0.89 accuracy — solid, but the weakest of the three.

Architecture — CNN (1D)

05 · Layer-by-layer: 1 input · 8 hidden · 1 output

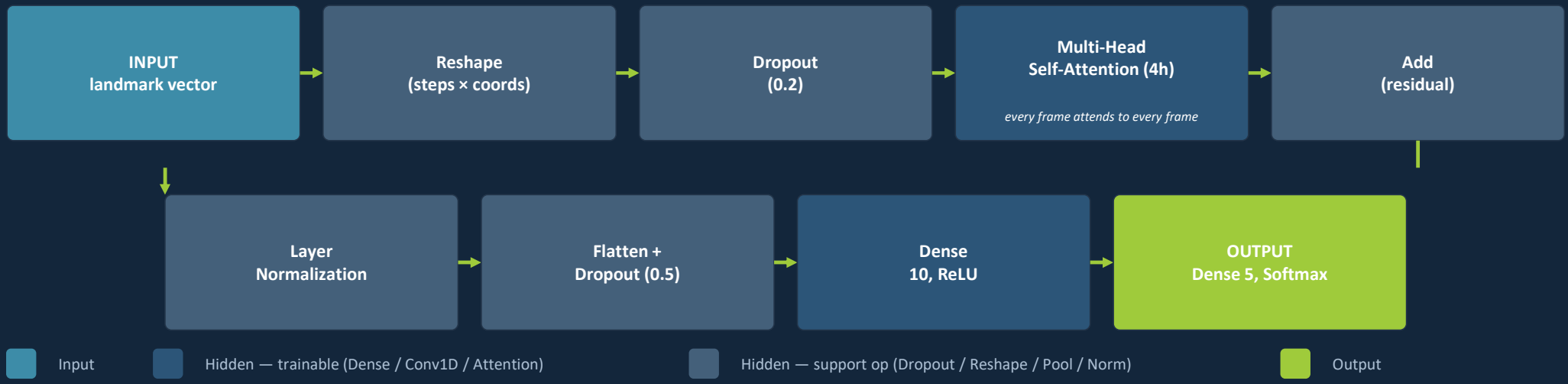


Why add convolutions

Reshape restores the time-step structure the FC model threw away. The two trainable Conv1D layers slide across nearby frames, learning local motion primitives (e.g. “fingers closing over 2–3 frames”) instead of memorizing a flat vector; pooling and flattening are non-trainable support ops that shrink and reformat the data between them. Result: 0.90 accuracy.

Architecture — Transformer

05 · Layer-by-layer: 1 input · 7 hidden · 1 output



Why attention wins here

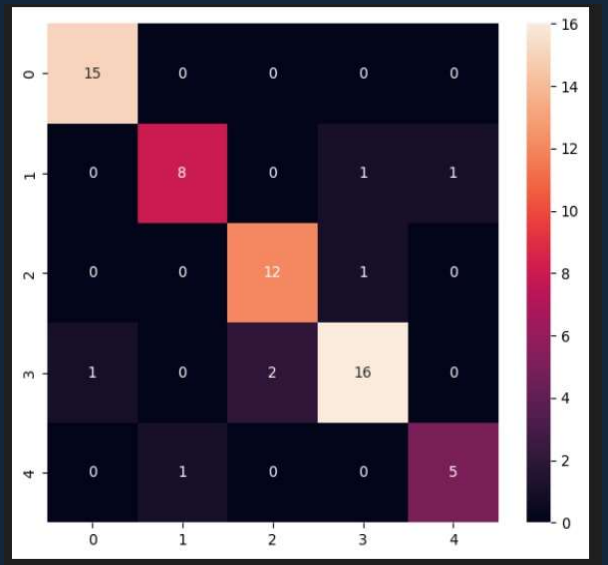
In plain terms: instead of treating every frame in the gesture equally, the model learns to "glance back" and give extra weight to whichever 1–2 frames actually decided the sign — like re-reading the one key word in a sentence instead of every word equally.

Technically: the only trainable hidden layers are the Multi-Head Attention block and the final Dense(10); Reshape / Add / LayerNorm / Flatten / Dropout carry data between them but learn nothing themselves. Result: 0.94 accuracy, the best of the three.

Results — Per-Model Evaluation

06 · Same 63 held-out samples, all three models

Fully-Connected (baseline)

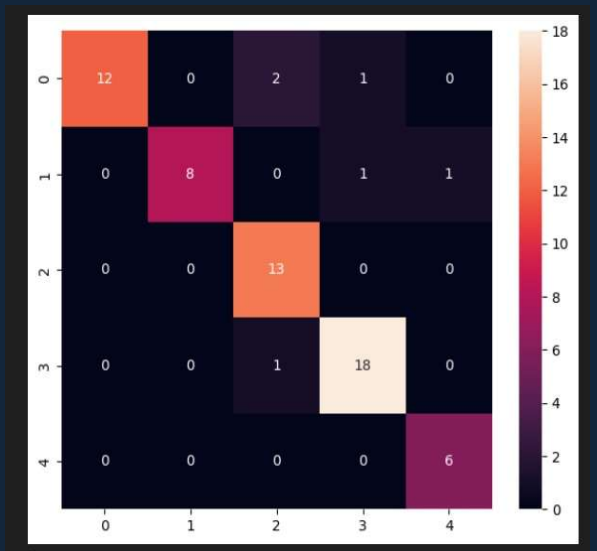


Classification Report

	precision	recall	f1-score	support
0	0.94	1.00	0.97	15
1	0.89	0.80	0.84	10
2	0.86	0.92	0.89	13
3	0.89	0.84	0.86	19
4	0.83	0.83	0.83	6
accuracy			0.89	63
macro avg	0.88	0.88	0.88	63
weighted avg	0.89	0.89	0.89	63

Accuracy: 0.89

CNN

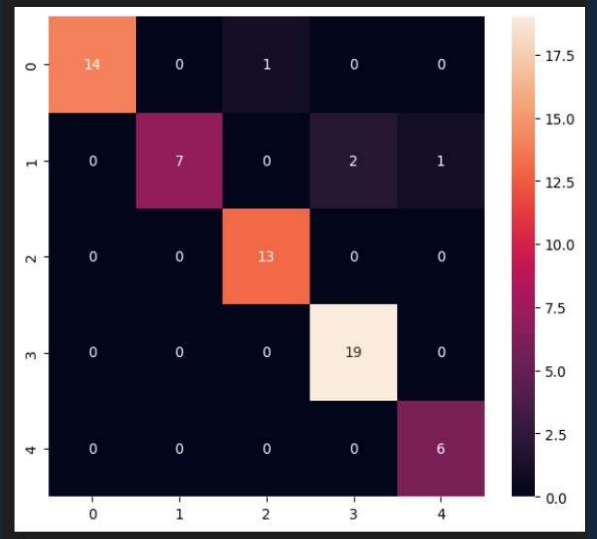


Classification Report

	precision	recall	f1-score	support
0	1.00	0.80	0.89	15
1	1.00	0.80	0.89	10
2	0.81	1.00	0.90	13
3	0.90	0.95	0.92	19
4	0.86	1.00	0.92	6
accuracy			0.90	63
macro avg	0.91	0.91	0.90	63
weighted avg	0.92	0.90	0.90	63

Accuracy: 0.90

Transformer



Classification Report

	precision	recall	f1-score	support
0	1.00	0.93	0.97	15
1	1.00	0.70	0.82	10
2	0.93	1.00	0.96	13
3	0.90	1.00	0.95	19
4	0.86	1.00	0.92	6
accuracy			0.94	63
macro avg	0.94	0.93	0.93	63
weighted avg	0.94	0.94	0.93	63

Accuracy: 0.94

Reading the Confusion Matrix

06 · What TP / TN / FP / FN actually mean here

For a given gesture class:

TP (True Positive)

The gesture was that class, and the model correctly said so.

FN (False Negative)

The gesture was that class, but the model missed it (said another class).

FP (False Positive)

The gesture was a different class, but the model wrongly called it this class.

TN (True Negative)

The gesture was not this class, and the model correctly did not call it this class.

The formulas

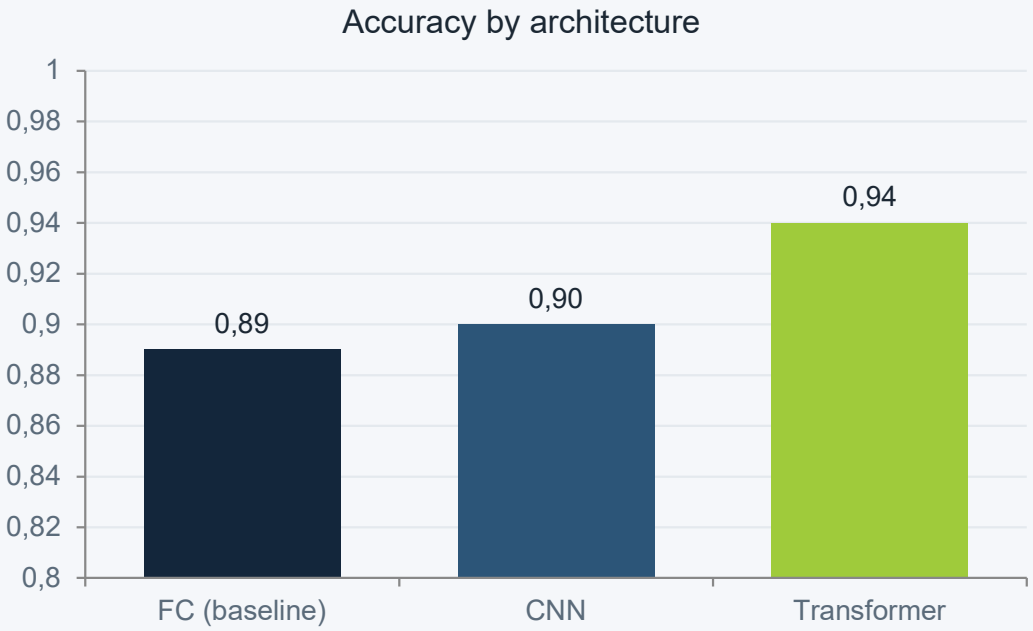
```
Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP) — of what we called this class, how much was right

Recall = TP / (TP + FN)   — of what actually was this class, how much we caught

F1 = 2 × (Precision × Recall) / (Precision + Recall)
```

Worked example — Transformer, gesture class 1

From our own confusion matrix: TP = 7, FN = 3, FP = 0, TN = 53 (out of 63 samples).
Precision = 7/7 = 1.00 · Recall = 7/10 = 0.70 · F1 = 0.82 — matches our classification report exactly.

Results — Head-to-Head

06 · Accuracy, Macro F1, Weighted F1



Model	Accuracy	Macro F1	Weighted F1
Fully-Connected (baseline)	0.89	0.88	0.89
CNN	0.90	0.90	0.90
Transformer	0.94	0.93	0.94

Accuracy vs. precision — which do we care about?

For safety gestures (STOP/EMERGENCY) recall matters most — a missed stop is dangerous. For general commands, precision avoids annoying false triggers. That's why we report both, not just accuracy.

Accuracy Isn't Everything

06 · Trade-offs — measured, not estimated

Metric (requested by Professor)	Average Value	Minimum	Maximum
Achievable Frame Rate	15.54 FPS	9.38 FPS	29.98 FPS
Camera Capture Latency	8.14 ms	1.60 ms	26.80 ms
MediaPipe Extraction Time	43.80 ms	13.90 ms	126.20 ms
TFLite Inference Time	0.19 ms	0.10 ms	0.80 ms
Overall Pipeline Latency	54.78 ms	18.10 ms	148.90 ms

We measured it: here's where the time actually goes.

The bottleneck isn't the classifier: MediaPipe's hand-detection step takes 43.80 ms on average — about 80% of our 54.78 ms total pipeline latency. The trained classifier itself responds in under 1 ms, which is negligible by comparison.

What this means for our accuracy-vs-speed trade-off: the accuracy differences between our three architectures barely matter for real-time performance, since classification time is a rounding error next to hand detection. The real lever for speed is upstream of model choice.

Measured on our current deployed classifier configuration in app.py. We recommend repeating this test for each of the three trained architectures individually to confirm the finding holds across all of them.

Embedded Deployment — Status & Decision

07 · Raspberry Pi: investigated, confirmed optional, not pursued

✓ **Investigated, discussed with Professor Geronazzo, and resolved — details below**

What we found

We investigated using a 64-bit Ubuntu virtual machine to simulate our target Raspberry Pi 5 deployment ahead of receiving the physical board. The VM correctly tests our Linux-side dependencies and OpenCV logic, but we hit a fundamental limitation: our development PCs use x86 processors, while the Raspberry Pi 5 uses ARM64. A VM cannot natively virtualize a different CPU architecture — emulating ARM64 on x86 requires translating every instruction on the fly, which severely degrades frame rate and makes the results unreliable for judging real-time performance or actual edge-hardware constraints.

Where we landed

- We raised this with Professor Geronazzo. He confirmed that physical Raspberry Pi 5 deployment and validation are **not mandatory** for this course — evaluating the algorithm itself, independent of the Pi, is what's required (see our accuracy, precision/recall/F1, and measured speed/latency results).

Given this, and the time available before the exam discussion, we made the informed decision not to pursue physical hardware deployment for this submission. It remains a possible extension if time allows.

Repository, Code & AI Use

08 · Reproducibility

GitHub Repository

- Structured repo with README, code, sample images, and evaluation results committed individually by each member.
- Documented build/run instructions, dependencies (MediaPipe, TensorFlow, OpenCV, scikit-learn) and .gitignore hygiene.
- Datasets linked externally rather than committed in full.

`github.com/giovannicamerotto-cmd/SignBridge_Camerotto_Brian_Faakhir`

Third-Party Code & AI Disclosure

- Built on an existing open-source hand-gesture-recognition project (kinivi/hand-gesture-recognition-mediapipe), cited in the README.
- Our contributions: extended gestures to all five fingers, added the CNN architecture option, added text-to-speech, captured our own industrial lexicon, retuned hyperparameters.
- AI assistants (Copilot, Gemini) were used for a large share of the coding scaffolding; every generated line was reviewed and corrected by the team, and we can explain each design choice.

References & Related Work

The more honestly we credit others, the clearer the value of our own work

- **MediaPipe Hands** — mediapipe.dev — real-time hand landmark detection backbone
- **Original implementation** — github.com/kinivi/hand-gesture-recognition-mediapipe — base gesture-recognition pipeline we adapted
- **Sign language reference** — handspeak.com — official ASL sign reference used while building our lexicon
- **Video tutorial** — youtu.be/7sywpZ7o2gg — walkthrough of the original pipeline
- **Our repository** — github.com/giovannicamerotto-cmd/SignBridge_Camerotto_Brian_Faakhir

Conclusion & Next Steps

Wrapping up

In short: a from-scratch sign-language system that lets a worker control a robot and inform colleagues in real time — with real, named trade-offs still to resolve before the factory floor.

What we achieved

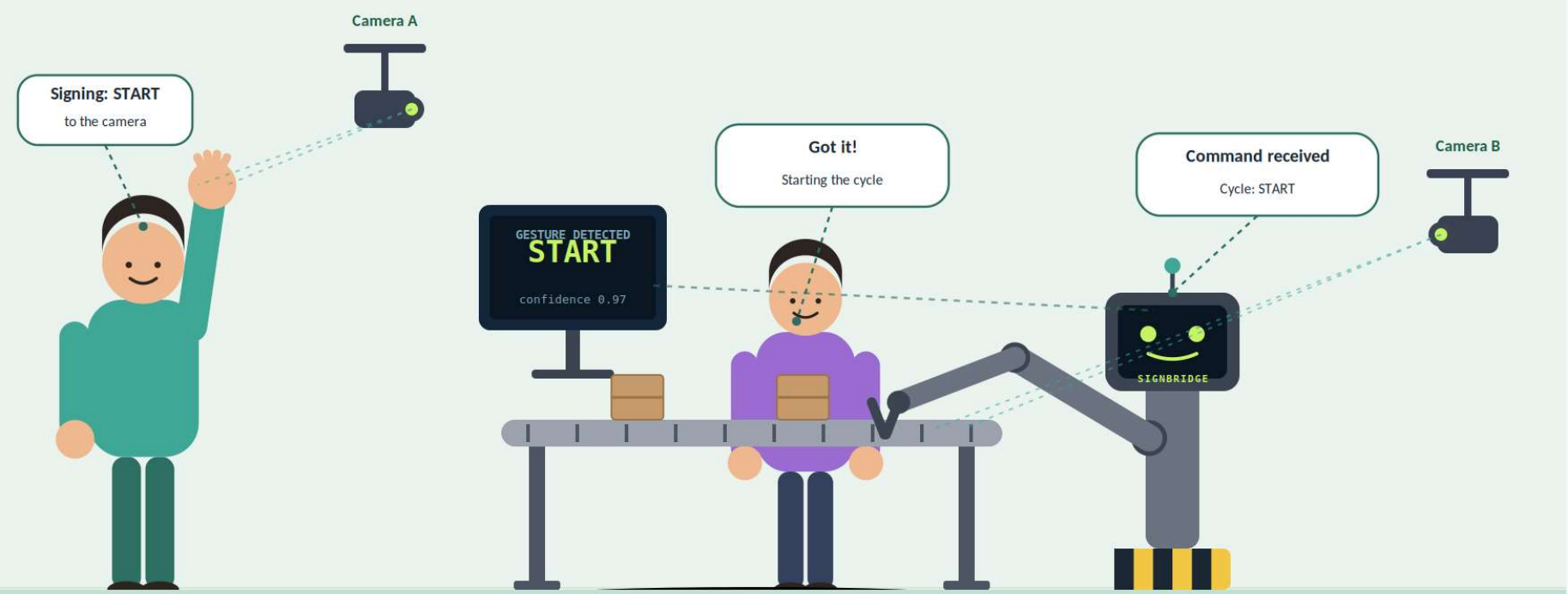
- A from-scratch dataset and documented ground truth for an industrial sign-language lexicon.
- A working real-time pipeline: hand tracking → gesture classification → robot/speech feedback, at ~9 FPS on ordinary laptop hardware.
- A fair, same-data comparison of 3 architectures (FC baseline, CNN, Transformer) using standard metrics — Transformer reaches 0.94 accuracy.
- Measured real end-to-end pipeline performance (camera capture, MediaPipe extraction, classifier inference) — confirmed the trained classifier itself is not the speed bottleneck; MediaPipe hand-detection is, at ~80% of total latency.

Honest limitations → next steps

- No traditional (non-deep-learning) baseline yet — next: add one for a complete comparison.
- Physical Raspberry Pi 5 deployment was investigated via VM but not pursued further — confirmed optional for this course by Professor Geronazzo, and out of scope given our timeline.
- Low-light / multi-angle robustness is planned but not yet tested — next: capture a small held-out test set under varied conditions.
- Dataset is currently just our own hands — next: grow lexicon and capture-group diversity.

Real-World Deployment Concept

A worker signs START — the camera, screen, robot, and colleagues all respond



Questions

SignBridge — Industrial Inclusion Through Computer Vision

University of Padua · Industrial Applications of Computer Vision

Stanis Brian · Faakhir · Giovanni Camerotto